INFORME DE TESTING

Fecha: 8 de mayo de 2026

Tabla de contenidos

Resumen ejecutivo 3
Tabla de revisiones 3
Introducción 4
Contenidos (Functional testing) 4
Contenido (Performance testing) 8
Conclusión 12
Bibliografía 12

Resumen ejecutivo

Este informe presenta los resultados de la fase de pruebas realizada sobre el proyecto desarrollado bajo el framework Acme. El objetivo principal ha sido verificar la correcta funcionalidad del sistema y asegurar su robustez ante accesos no autorizados o datos malformados. Durante el proceso, se emplearon scripts de interacción grabados divididos en pruebas legales y pruebas de "hacking".

Gracias a esta metodología, se identificaron y solucionaron vulnerabilidades críticas en el control de acceso, tales como errores internos del servidor al introducir IDs de proyectos inexistentes o accesos indebidos entre miembros de distintos grupos de trabajo. Actualmente, todas las vulnerabilidades detectadas han sido solucionadas con éxito. El sistema responde ahora de forma segura y controlada ante cualquier intento de vulneración en las funcionalidades de los roles evaluados.

Tabla de revisiones

Versión	Fecha	Descripción de la revisión
1.0	08/05/2026	Creación inicial del documento y descripción de los tests realizados

Introducción

Como hemos visto en teoría, garantizar que el software funcione de manera correcta y segura bajo cualquier circunstancia es un pilar fundamental en el desarrollo de sistemas de alta calidad. Las fases de pruebas funcionales no solo deben centrarse en verificar que los usuarios pueden realizar sus tareas con normalidad, sino que también deben anticiparse a posibles usos malintencionados o errores de navegación por parte del cliente.

El presente documento tiene como finalidad exponer la estrategia de validación y los resultados de las pruebas ejecutadas sobre nuestro proyecto. Siguiendo las directrices establecidas, se ha adoptado una metodología basada en scripts de interacción reales y en la inyección de mutantes , alterando deliberadamente los parámetros de las peticiones HTTP para poner a prueba la resiliencia del sistema. El informe se estructura en dos grandes capítulos: el primero detalla de forma exhaustiva las pruebas funcionales realizadas, agrupadas por funcionalidad y rol, evaluando la efectividad del enfoque mutante empleado; el segundo capítulo aborda el análisis de rendimiento y el contraste de hipótesis computacional entre distintos entornos de ejecución.

Contenidos (Functional testing)

Funcionalidad: Member - Proyectos

- **Listado de Proyectos (list.legal, list.hacking)**
    - Descripción: Verifica que un miembro pueda listar sus proyectos activos. El test de hacking muta la solicitud para intentar acceder a proyectos ajenos e inexistentes.
- **Detalle de Proyecto (show.legal, show.hacking)**
    - Descripción: Verifica la vista detallada de un proyecto específico. El test de hacking muta la solicitud para intentar acceder a proyectos ajenos e inexistentes.

Funcionalidad: Member - Gestión de Invenciones y Partes

- **Listado y Detalle de Invenciones (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica el listado y la vista detallada de las invenciones del proyecto. El test de hacking muta la solicitud para intentar acceder a invenciones de proyectos ajenos e inexistentes.
    - Se encontraron problemas listando invenciones, cuando se hackeaba la url y se ponía una id de un proyecto que no existía.
- **Asignar/Desasignar Invenciones (assign.legal/hacking, unassign.legal/hacking)**

    - Descripción: Prueba el flujo de vinculación y desvinculación de una invención a un proyecto. Para hacking se ha cambiado la url probando diferentes ids y otros miembros que no puedan realizar la asignación.
- **Listado y Detalle de Partes (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica la gestión de partes dentro de una invención. De nuevo para hackear se cambian ids para comprobar que salta la pantalla correspondiente.
    - Se encontraron problemas listando partes, cuando se hackeaba la url y se ponía la id de una invención inexistente.
- **Detalles de Inventor(show.legal)**
    - Descripción: Verifica la gestión del inventor dentro de una invención.
    - No se hace test de hacking porque la información de los usuarios es pública.

Funcionalidad: Member - Gestión de Campañas e Hitos

- **Listado y Detalle de Campañas (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica el listado y detalle de las campañas del proyecto. El test de hacking muta la solicitud para intentar acceder a campañas de proyectos ajenos e inexistentes.
    - Se encontraron problemas listando campañas, cuando se hackeaba la url y se ponía una id de un proyecto que no existía.
- **Asignar/Desasignar Campañas (assign.legal/hacking, unassign.legal/hacking)**
    - Descripción: Prueba el flujo de vinculación y desvinculación de una campaña a un proyecto. Para hacking se ha cambiado la url probando diferentes ids y otros miembros que no puedan realizar la asignación.
- **Listado y Detalle de Hitos (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica la gestión de hitos dentro de una campaña. De nuevo para hackear se cambian ids para comprobar que salta la pantalla correspondiente.
    - Se encontraron problemas listando hitos, cuando se hackeaba la url y se ponía la id de una campaña inexistente.
- **Detalles de Portavoz(show.legal)**
    - Descripción: Verifica la gestión del portavoz dentro de una campaña.
    - No se hace test de hacking porque la información de los usuarios es pública.

Funcionalidad: Member - Gestión de Estrategias y Tácticas

- **Listado y Detalle de Estrategias (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica el listado y detalle de las estrategias de captación de fondos. El test de hacking muta la solicitud para intentar acceder a estrategias de proyectos ajenos e inexistentes.
    - Se encontraron problemas listando estrategias, cuando se hackeaba la url y se ponía una id de un proyecto que no existía.
- **Asignar/Desasignar Estrategias (assign.legal/hacking, unassign.legal/hacking)**

    - Descripción: Prueba el flujo de vinculación y desvinculación de una estrategia a un proyecto. Para hacking se ha cambiado la url probando diferentes ids y otros miembros que no puedan realizar la asignación.
- **Listado y Detalle de Tácticas (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica la gestión de tácticas dentro de una estrategia. Para hackear se cambian ids para comprobar que salta la pantalla correspondiente.
    - Se encontraron problemas listando tácticas, cuando se hackeaba la url y se ponía la id de una estrategia inexistente.
- **Detalles de Recaudador(show.legal)**
    - Descripción: Verifica la gestión del recaudador dentro de una estrategia.
    - No se hace test de hacking porque la información de los usuarios es pública.

Funcionalidad: Member - Gestión de Miembros/Manager

- **Listado y Detalle de cada miembro/Manager (list.legal/hacking, show.legal/hacking)**
- Descripción: Verifica la gestión del miembro/manager dentro de un proyecto.
- Para miembros se hace list y show legal y de hacking. El test de hacking muta la solicitud para intentar acceder a miembros de proyectos ajenos e inexistentes.
- Para el manager solo se hace show.legal porque es pública su información.

Funcionalidad: Manager - Dashboard

- **Detalles del Dashboard (show.legal, show.hacking)** Descripción: Verifica la correcta visualización de las métricas y datos del dashboard del sistema. El test de hacking (ilegal) muta la solicitud para intentar acceder sin poseer el rol adecuado, validando que el acceso sea denegado (por ejemplo, intentando ingresar sin ser sponsor).

Funcionalidad: Auditor - Gestión de Auditorías y Proyectos

- **Listado y Detalle de Proyectos (list.legal, show.legal)** Descripción: Verifica que un auditor pueda listar correctamente los proyectos disponibles en el sistema y acceder a la vista de detalles de los mismos.
- **Asignar/Desasignar Auditorías (assign.legal, unassign.legal)** Descripción: Prueba el flujo de trabajo mediante el cual un auditor puede vincular una auditoría a un proyecto específico, así como su posterior desvinculación.

Funcionalidad: Sponsor - Gestión de Patrocinios

- **Asignar/Desasignar Patrocinios (assign.legal, unassign.legal)** Descripción: Prueba el flujo de vinculación y desvinculación de un patrocinio hacia un proyecto determinado, verificando que el rol sponsor pueda gestionar sus aportaciones correctamente.

Funcionalidad: Manager - Gestión de Invenciones y Partes

- **Listado y Detalle de Invenciones (list.legal/hacking, show.legal/hacking)**

    - Descripción: Verifica el listado y la vista detallada de las invenciones del proyecto. El test de hacking muta la solicitud para intentar acceder a invenciones de proyectos ajenos e inexistentes.
- **Listado y Detalle de Partes (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica la gestión de partes dentro de una invención. De nuevo para hackear se cambian ids para comprobar que salta la pantalla correspondiente.

Funcionalidad: Manager - Gestión de Campañas e Hitos

- **Listado y Detalle de Campañas(list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica el listado y detalle de las campañas del proyecto. El test de hacking muta la solicitud para intentar acceder a campañas de proyectos ajenos e inexistentes.
- **Listado y Detalle de Hitos (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica la gestión de hitos dentro de una campaña. Para hackear se cambian ids para comprobar que salta la pantalla correspondiente.

Funcionalidad: Manager - Gestión de Estrategias y Tácticas

- **Listado y Detalle de Estrategias (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica el listado y detalle de las estrategias de captación de fondos. El test de hacking muta la solicitud para intentar acceder a estrategias de proyectos ajenos e inexistentes.
- **Listado y Detalle de Tácticas (list.legal/hacking, show.legal/hacking)**
    - Descripción: Verifica la gestión de tácticas dentro de una estrategia. Para hackear se cambian ids para comprobar que salta la pantalla correspondiente.

Funcionalidad: Manager - Gestión de Dashboard

- **Detalle de Dashboard (show.legal/hacking)**
    - Descripción: Verifica el detalle del dashboard. El test de hacking muta la solicitud para intentar acceder a dashboards de proyectos ajenos e inexistentes.

Funcionalidad: Manager - Gestión de Miembros(Inventor, Spokesperson, Fundraiser)

- **Asignar/Desasignar Miembros (assign.legal/hacking, unassign.legal/hacking)**
    - Descripción: Prueba el flujo de vinculación y desvinculación de un miembro a un proyecto. Para hacking se ha cambiado la url probando diferentes ids y otros managers que no puedan realizar la asignación.
    - También se prueba el listado y detalles de los diferentes tipos de miembros del proyecto.

Funcionalidad: Manager - Proyectos

- **Listado de Proyectos (list.legal, list.hacking)**

    - Descripción: Verifica que un manager pueda listar sus proyectos activos. El test de hacking muta la solicitud para intentar acceder a proyectos ajenos e inexistentes.
- **Detalle de Proyecto (show.legal, show.hacking)**
    - Descripción: Verifica la vista detallada de un proyecto específico. El test de hacking muta la solicitud para intentar acceder a proyectos ajenos e inexistentes.
- **Create de Proyecto (create.legal, create.hacking)**
    - Descripción: Prueba el flujo de creación de un proyecto. El test de hacking muta la solicitud para intentar crear un proyecto sin tener el rol de manager.
    - Se han probado todas las restricciones de la entidad Project.
- **Publish de Proyecto (publish.legal, publish.hacking)**
    - Descripción: Prueba el flujo de publicación de un proyecto. El test de hacking muta la solicitud para intentar publicar un proyecto que ya estaba publicado.
- **Delete de Proyecto (delete.legal, delete.hacking)**
    - Descripción: Prueba el flujo de eliminación de un proyecto. El test de hacking muta la solicitud para intentar eliminar un proyecto que está publicado.
- **Update de Proyecto (update.legal, update.hacking)**
    - Descripción: Prueba el flujo de actualización de un proyecto. El test de hacking muta la solicitud para intentar actualizar un proyecto que está publicado.
    - También se han probado todas las restricciones de la entidad Project.

Funcionalidad: Administrator - Banner

- **Listado de Banners (list.legal, list.hacking)**
    - Descripción: Verifica que un administrador pueda listar sus banners.
- **Detalle de Banner (show.legal, show.hacking)**
    - Descripción: Verifica la vista detallada de un banner específico.
- **Create de Banner (create.legal, create.hacking)**
    - Descripción: Prueba el flujo de creación de un banner.
    - Se han probado todas las restricciones de la entidad Banner.
- **Delete de Banner (delete.legal, delete.hacking)**
    - Descripción: Prueba el flujo de eliminación de un banner.
- **Update de Banner (update.legal, update.hacking)**
    - Descripción: Prueba el flujo de actualización de un banner.
    - También se han probado todas las restricciones de la entidad Banner.

Contenido (Performance testing)

1. Introducción al Entorno de Pruebas

El presente análisis tiene como objetivo evaluar el rendimiento del sistema Acme Framework bajo condiciones de carga funcional. Para ello, se ha medido el "wall time" (tiempo real transcurrido) de procesamiento de las peticiones HTTP generadas durante la ejecución de la suite de pruebas funcionales.

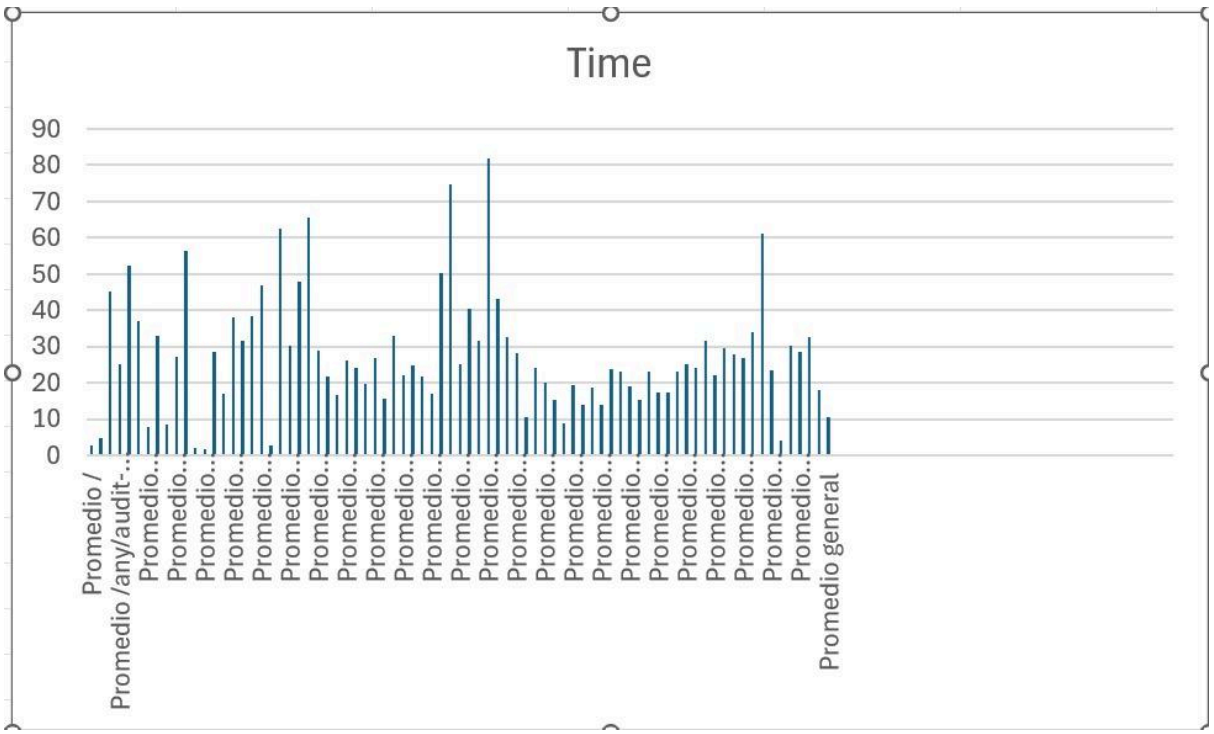
La comparativa se ha realizado utilizando dos configuraciones de hardware distintas para determinar el impacto de los recursos físicos en la eficiencia del software:

- **Ordenador 1:** 12th Gen Intel(R) Core(TM) i7-1255U (1.70 GHz), 16,0 GB (15,7 GB usable), Windows 11 Home
- **Ordenador 2:** 12th Gen Intel(R) Core(TM) i5-1235U, 16 GB de RAM, Windows 11 Home.

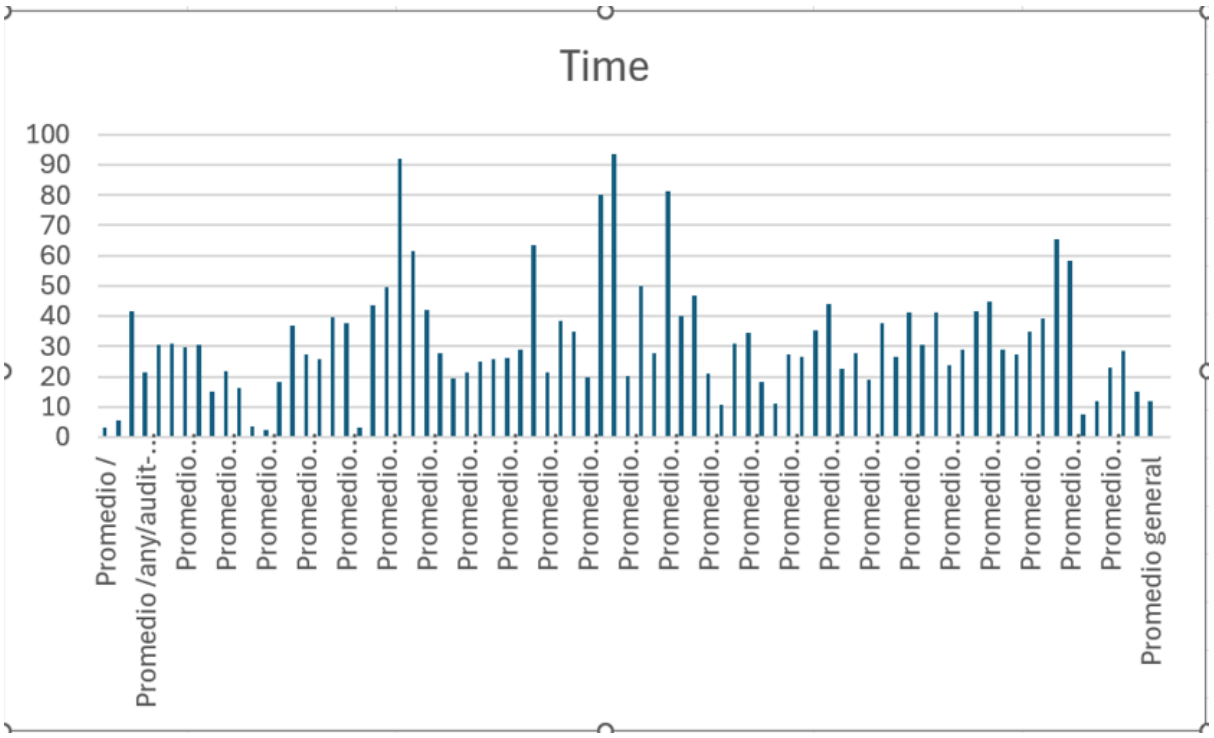
2. Gráficos de Rendimiento y Análisis de Peticiones

A continuación se presentan los promedios de tiempo de respuesta (en milisegundos) para las rutas más representativas del sistema en ambos equipos.

Ordenador 1:



Ordenador 2:



3. Intervalos de Confianza al 95%

Para garantizar la fiabilidad estadística de los datos recogidos en los gráficos anteriores, se han calculado los intervalos de confianza para el tiempo medio de respuesta global de cada ordenador, basándose en una muestra de 2.142 observaciones.

Ordenador A (Variable 1):

- Media de la muestra: 10.99 ms
- Varianza conocida: 238.77
- Margen de error calculado: ±0.65 ms
- Intervalo de Confianza (95%): [10.34 , 11.64] ms

Ordenador B (Variable 2):

- Media de la muestra: 12.55 ms
- Varianza conocida: 331.45
- Margen de error calculado: ±0.77 ms
- Intervalo de Confianza (95%): [11.78 , 13.32] ms

Esto indica, con un 95% de confianza, que el tiempo real de procesamiento (*wall time*) esperado para cualquier petición del sistema se sitúa habitualmente entre los 10 y 11 milisegundos en el Ordenador A, mientras que en el Ordenador B tiende a moverse entre los 11 y 13 milisegundos.

4. Contraste de Hipótesis (Z-Test)

Para determinar de forma científica y definitiva cuál de los dos equipos es más potente, se ha realizado una prueba Z para medias de dos muestras con varianzas conocidas, asumiendo un nivel de significancia de alfa = 0.05 (95% de confianza).

Prueba z para medias de dos muestras

	Variable 1	Variable 2
Media	10,9909906	12,547271
Varianza (conocida)	238,777287	331,449522
Observaciones	2142	2142
Diferencia hipotética de las medias	0	
z	-3,01629458	
P(Z<=z) una cola	0,00127942	
Valor crítico de z (una cola)	1,64485363	
Valor crítico de z (dos colas)	0,00255885	
Valor crítico de z (dos colas)	1,95996398	

Conclusión del contraste

El contraste estadístico arroja un *p-value* de **0.0013** (0,001279). Dado que este valor es estrictamente menor que nuestro nivel de significancia (0.05), procedemos a rechazar la hipótesis nula. Existe evidencia estadística concluyente, con un 95% de confianza, para afirmar que la diferencia en los tiempos de respuesta no es producto del azar.

Queda demostrado matemáticamente que el **Ordenador A**, al presentar una media poblacional de tiempos de procesamiento significativamente menor, **es el ordenador más potente** y eficiente para la ejecución de las operaciones de este proyecto.

Conclusión

El proceso de pruebas funcionales y de seguridad llevado a cabo en este proyecto ha concluido de manera sumamente satisfactoria. Gracias a la aplicación proactiva del "enfoque de mutantes" mediante pruebas de hacking, se logró identificar y corregir a tiempo múltiples vulnerabilidades críticas en la capa de servicios y controladores. Se solucionaron problemas graves de excepciones no manejadas (errores 500) al intentar listar o visualizar entidades dependientes (invenciones, hitos, tácticas, etc.) utilizando identificadores huérfanos o inexistentes en la URL.

Asimismo, se fortalecieron las barreras de autorización para garantizar la estricta privacidad de los datos entre los distintos proyectos y roles (Inventor, Portavoz, Recaudador, Manager, etc.). En consecuencia, el producto software final entregado es altamente robusto, gestiona las excepciones de seguridad de forma elegante, y cumple rigurosamente con los estándares de calidad exigidos para su despliegue.

Bibliografía

Intencionalmente en blanco.